

HOMEWORK #1

Artificial Intelligence

Name: Student ID:
Major: Electronic Science and Technology

Date: 2025 年 3 月 22 日

Problem 1.

Give a complete problem formulation for each of the following. Choose a formulation that is precise enough to be implemented.

- a) Using only four colors, you have to color a planar map in such a way that no two adjacent regions have the same color.
- b) A 3-foot-tall monkey is in a room where some bananas are suspended from the 8-foot ceiling. He would like to get the bananas. The room contains two stackable, movable, climbable 3-foot-high crates.
- c) You have a program that outputs the message “illegal input record” when fed a certain file of input records. You know that processing of each record is independent of the other records. You want to discover what record is illegal.
- d) You have three jugs, measuring 12 gallons, 8 gallons, and 3 gallons, and a water faucet. You can fill the jugs up or empty them out from one to another or onto the ground. You need to measure out exactly one gallon.

Answer :

a: Four-Color Map Problem

Initial State: All regions are uncolored.

Successor function: Assign a color to an uncolored region, ensuring that the chosen color is different from the colors of all adjacent regions.

Goal Test: All regions are colored, and no two adjacent regions share the same color.

Cost Function: Assume the cost of coloring a region is 1. The total cost is the number of regions, n .

b: Monkey and Bananas Problem

Initial State: The monkey is on the ground, the two crates are on the ground, and the bananas have not been reached:

Successor function: 1. Move the monkey to a crate or the ground. 2. Move a crate to a new position. 3. Stack one crate on top of the other. 4. Unstack one crate from the other. 5. Climb a crate. 6. Grab the bananas.

Goal Test: The monkey reaches the bananas:

Cost Function: Assume each action has a cost of 1. The total cost is the number of actions taken to reach the goal.

c: Illegal Input Record Problem

Initial State: The full set of input records.

Successor function: 1. Divide the current set of records into smaller subsets. 2. Test each subset to determine if it contains the illegal record. 3. Discard subsets that do not contain the illegal record.

Goal Test: Find the single illegal record.

Cost Function: The cost is the number of tests performed. The goal is to minimize the number of tests.

State Space: Each state represents the amount of water in the three jugs. A state can be represented as a tuple (x,y,z) , where: - x : The amount of water in the 12-gallon jug. - y : The amount of water in the 8-gallon jug. - z : The amount of water in the 3-gallon jug.

Initial State: $(0,0,0)$, where all jugs are empty.

Successor function: 1. Fill a jug completely from the faucet. 2. Empty a jug onto the ground. 3. Pour water from one jug into another until either the first jug is empty or the second jug is full.

Goal Test: The goal is to measure exactly 1 gallon of water in any of the jugs. For example, a goal state could be $(1,y,z)$, $(x,1,z)$, or $(x,y,1)$.

Cost Function: Assume each action has a cost of 1. The total cost is the number of actions taken to reach the goal.

Problem 2.

Consider a state space where the start state is number 1 and each state has two successors: numbers $2k$ and $2k + 1$.

- Draw the portion of the state space for states 1 to 15.
- Suppose the goal state is 11. List the order in which nodes will be visited for breadth-first search, depth-limited search with limit 3, and iterative deepening search.
- How well would bidirectional search work on this problem? What is the branching factor in each direction of the bidirectional search?
- Does the answer to (c) suggest a reformulation of the problem that would allow you to solve the problem of getting from state 1 to a given goal state with almost no search?
- Call the action going from k to $2k$ Left, and the action going to $2k + 1$ Right. Can you find an algorithm that outputs the solution to this problem without any search at all?

Answer :

- a: The figure of the state space for states 1 to 15 is shown below:

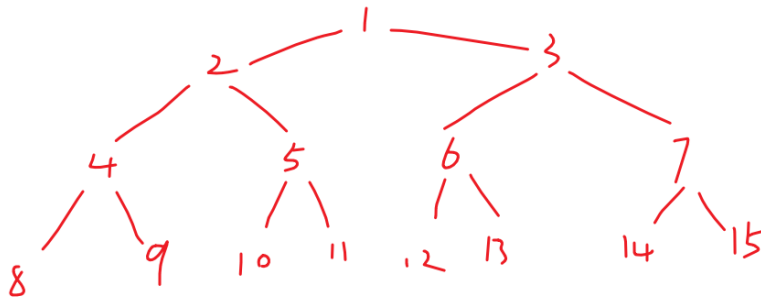


图 1 State Space for States 1 to 15

- b: The order in which nodes will be visited for each search method is as follows:

BFS Order: 1,2,3,4,5,6,7,8,9,10,11.

Depth-Limited Search with limit 3 Order: 1,2,4,8,9,5,10,11.

Iterative Deepening Search Order:

Depth 0: 1

Depth 1: 1,2,3

Depth 2: 1,2,4,5,6,7

Depth 3: 1,2,4,8,9,5,10,11.

c:

Bidirectional search work well on this problem because the goal state is known. In the reverse direction, the successors of a state n are $\lfloor n/2 \rfloor$, only one successor, so it is easy to find the path from the goal state to the start state.

The branching factor in forward direction is 2 and in reverse direction is 1.

d:

Yes, start from the goal state and apply the reverse actions (Left and Right) to reach the start state. This way, the problem of getting from state 1 to a given goal state can be solved with almost no search.

e:

Yes, the solution can be found without search by using the binary representation of the goal state. The algorithm is as follows:

1. Convert the goal state k into its binary representation.
2. Starting from the second bit (ignoring the leading 1), interpret each bit:
 - 0: Take the "Left" action.
 - 1: Take the "Right" action.
3. Output the sequence of actions.

Problem 3.

Consider the two-player game described in Figure 5.17

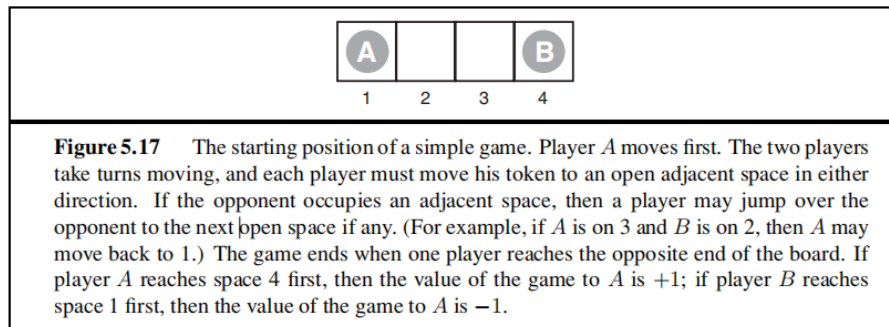


图 2 figure 5.17

- a) Draw the complete game tree, using the following conventions:
 - Write each state as (s_A, s_B) , where s_A and s_B denote the token locations.
 - Put each terminal state in a square box and write its game value in a circle.
 - Put loop states (states that already appear on the path to the root) in double square boxes. Since their value is unclear, annotate each with a "?" in a circle.
- b) Now mark each node with its backed-up minimax value (also in a circle). Explain how you handled the "?" values and why.
- c) Explain why the standard minimax algorithm would fail on this game tree and briefly sketch how you might fix it, drawing on your answer to (b). Does your modified algorithm give optimal decisions for all games with loops?
- d) This 4-square game can be generalized to n squares for any $n > 2$. Prove that A wins if n is even and loses if n is odd.

Answer :

a: The complete game tree is shown below:

b:

If all the successors have "?" values, it's obvious that the parent node will also have a "?" value.

Otherwise, the backed-up minimax value of a node is the maximum value of its successors if it is a MAX node, and the minimum value of its successors if it is a MIN node.

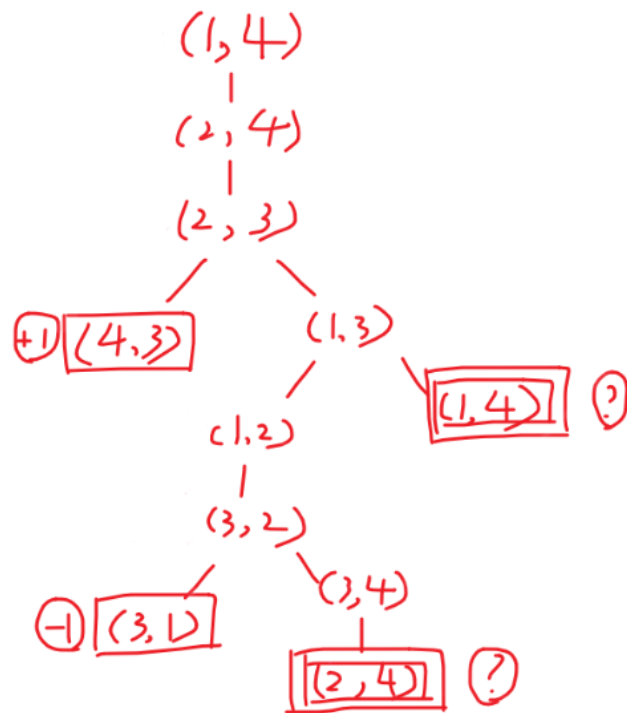


图 3 Game Tree

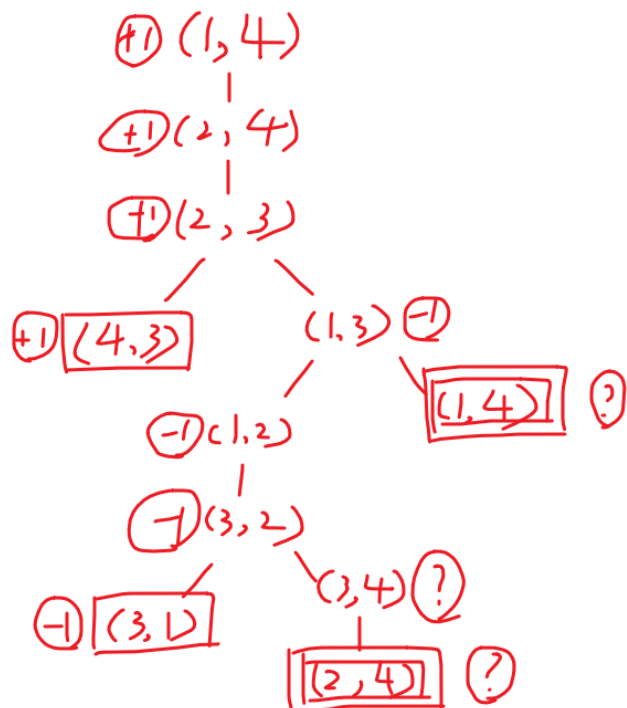


图 4 complete game tree

c:

1.The standard minimax algorithm fails on this game tree because it does not account for loops. When a loop is encountered, the algorithm may enter infinite recursion or fail to assign a value to the loop state.

2.To handle loops, the algorithm must detect repeated states and assign them a value based on the minimax principle:

- Use a visited set to track states that have already been evaluated.
- When a loop is detected, assign a "?" value
- handled the "?" values as (b) described.

3.The modified algorithm provides optimal decisions for games with loops as long as the loop values are assigned consistently based on the minimax principle. However, the solution may not be optimal if the loop values are not well-defined or if the game has stochastic elements.

d:

We can prove this by induction on n :

Base case: $n = 3$, A loses. This can be verified by examining the game tree.

Inductive step: Assume when $n = 2k + 1$ ($k > 1$), A loss

When $n = 2k + 3$, it is $[1, 2, 3 \dots, 2k+3]$. At first A in 1 and only move to 2, B in $2k+3$ and only move to $2k+2$. Now it begin a subgame in $[2, 3 \dots, 2k+2]$. According to the induction hypothesis, A loss in $[2, 3 \dots, 2k+2]$. So no matter how A move, B can always move to 2 before A move to $2k+2$. When B in 2, if A not in 1, B win with move to 1. If A in 1, B win with move to 3, A to 2, B to 1.

Thus if when $n = 2k + 1$ ($k > 1$), A loss. Then A also loss when $n = 2k + 3$. According to the induction, A loss when n is odd.

Same as above, when $n = 4$, A wins. It can be verified by examining the game tree.

Similarly, we can prove that when $n = 2k + 2$ ($k > 1$), A wins.

Above all, A wins if n is even and loses if n is odd.

Problem 4.

Consider the problem of solving two 8-puzzles.

- a) How large is the reachable state space? Give an exact numerical expression.
- b) Suppose we make the problem adversarial as follows: the two players take turns moving; a coin is flipped to determine the puzzle on which to make a move in that turn; and the winner is the first to solve one puzzle. Which algorithm can be used to choose a move in this setting?
- c) Give an informal proof that someone will eventually win if both play perfectly.

Answer :

a:

For a single 8-puzzle, the total number of possible arrangements of the 9 tiles (8 numbered tiles and 1 blank space) is $9!$. However, only half of these arrangements are solvable, so the reachable state space for a single 8-puzzle is:

$$\frac{9!}{2} = 181,440$$

For two independent 8-puzzles, the state space is the Cartesian product of the state spaces of the two puzzles. Therefore, the total reachable state space is:

$$\left(\frac{9!}{2}\right)^2 = 181,440^2$$

b:

In this case, the random nature of the game makes it a stochastic game. The algorithm that can be used to choose a move in this setting is the Expectiminimax algorithm.

c:

For the Initial two 8-puzzles, it is Non-competitive game, and the two players can play perfectly to solve they own puzzle, the state space is finite and ever player make the best move, the game will eventually end.

But for the two-player adversarial 8-puzzle game in (b), the game is competitive and stochastic. The game may not finish.

consider the following:

A player toss a coin to determine he can move in the 1st puzzle and the 1st puzzle will be solved with 2 moves. In this case, if A choice to move towards the goal, we can calculate the probability of A win is $1/3$ and B win is $2/3(1/2+1/8+1/32+\dots=2/3)$.

So, in this case, by the Expectiminimax algorithm, A should not move towards the goal.

In order to finish the game, it always come with the case shown below, and the perfectly play will not choice to move towards the goal. So, the game will not eventually end.